# Image Classification



Demonstrate **Ultralytics** using Python: Image classification effects on images, videos, and real-time detection.

## 1. Model Introduction

Image classification is the simplest of the three tasks, involving classifying an entire image into one of a set of predefined categories.

The output of an image classifier is a single class label and a confidence score. Image classification is very useful when you only need to know which category an image belongs to, without needing to know the location or exact shape of objects in that category.

## 2. Image Classification: Image

Use yolo26n-cls_ncnn_model to predict images in the ultralytics26 folder (not images that come with ultralytics).

Enter the code folder:

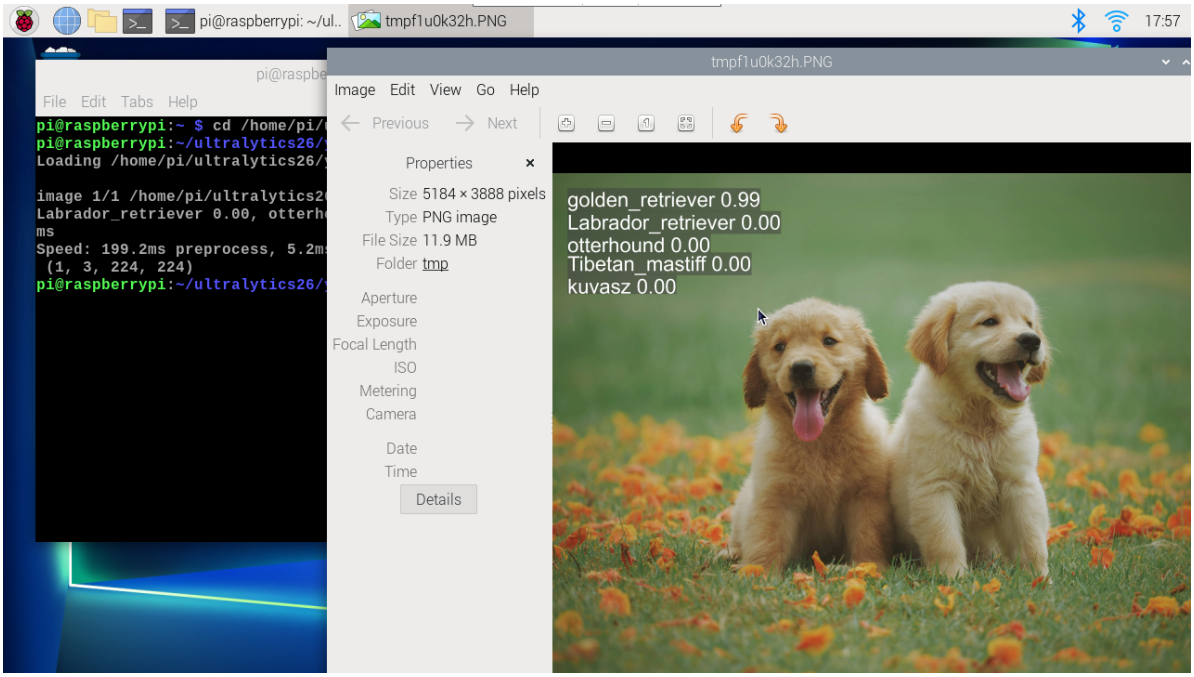
```
cd /home/pi/ultralytics26/yahboom_demo
```

Run the code:

```
python3 04.classification_image.py
```

### Effect Preview

yolo recognition output image location: /home/pi/ultralytics26/output/




Example code:

```
from ultralytics import YOLO

# Load a model
#model = YOLO("/home/pi/ultralytics26/yolo26n-cls.pt")
model = YOLO("/home/pi/ultralytics26/yolo26n-cls_ncnn_model")

# Run batched inference on a list of images
results = model("/home/pi/ultralytics26/assets/dog.jpg")  # return a list of
Results objects

# Process results list
for result in results:
    # boxes = result.boxes  # Boxes object for bounding box outputs
    # masks = result.masks  # Masks object for segmentation masks outputs
    # keypoints = result.keypoints  # Keypoints object for pose outputs
    probs = result.probs  # Probs object for classification outputs
    # obb = result.obb  # Oriented boxes object for OBB outputs
    result.show()  # display to screen
    result.save(filename="/home/pi/ultralytics26/output/dog_output.jpg")  # save
to disk
```

# 3. Image Classification: Video

Use yolo26n-cls_ncnn_model to predict videos in the ultralytics26 folder (not videos that come with ultralytics).

Enter the code folder:

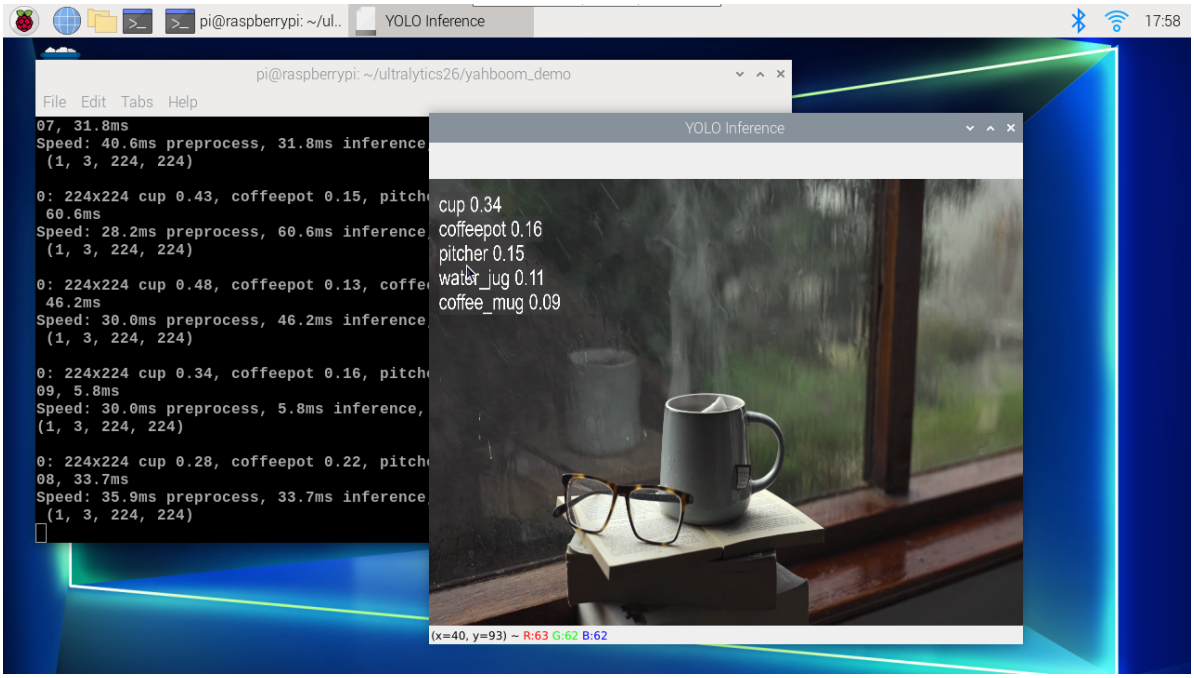
```
cd /home/pi/ultralytics26/yahboom_demo
```

Run the code:

```
python3 04.classification_video.py
```

## Effect Preview

yolo recognition output video location: /home/pi/ultralytics26/output/




Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
#model = YOLO("/home/pi/ultralytics26/yolo26n-cls.pt")
model = YOLO("/home/pi/ultralytics26/yolo26n-cls_ncnn_model")

# Open the video file
video_path = "/home/pi/ultralytics26/videos/cup.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/pi/ultralytics26/output/04.cup_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
```

```
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

# 4. Image Classification: Real-time Detection

## 4.1 USB Camera

Use yolo26n-cls_ncnn_model to predict USB camera footage.

Enter the code folder:

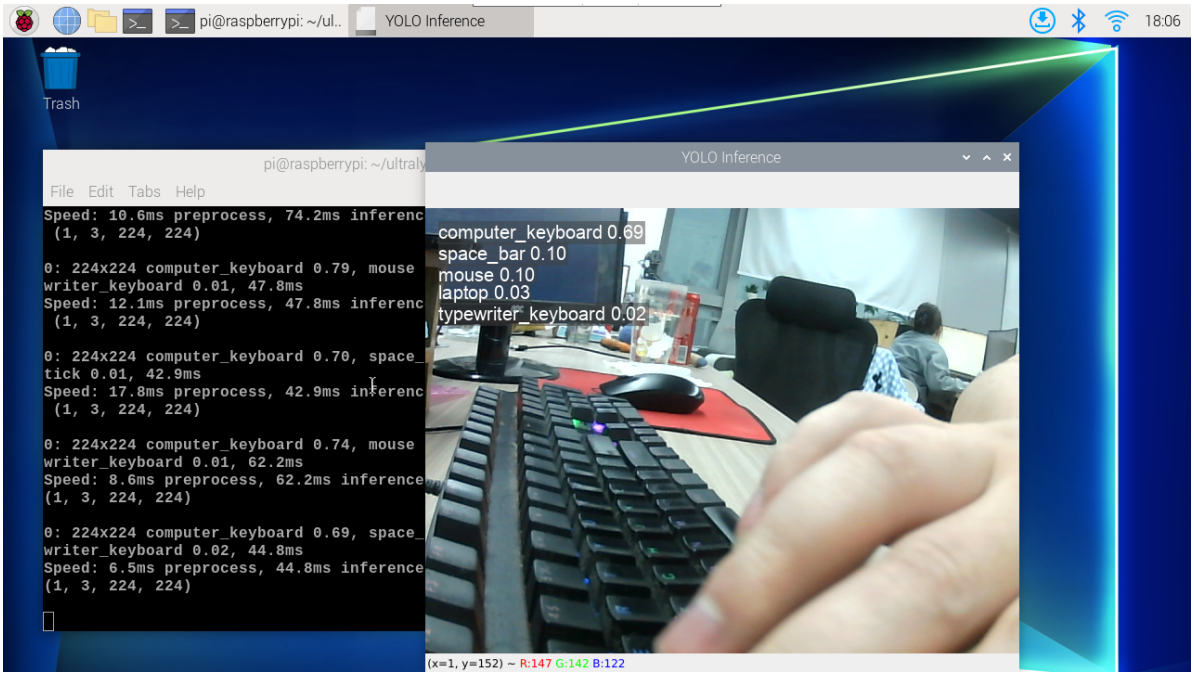
```
cd /home/pi/ultralytics26/yahboom_demo
```

Run the code:

```
python3 04.classification_camera_usb.py
```

### Effect Preview

yolo recognition output video location: /home/pi/ultralytics26/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
#model = YOLO("/home/pi/ultralytics26/yolo26n-cls.pt")
model = YOLO("/home/pi/ultralytics26/yolo26n-cls_ncnn_model")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter.fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/pi/ultralytics26/output/04.classification_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

## 4.2 CSI Camera

Use yolo26n-cls_ncnn_model to predict CSI camera footage.

Enter the code folder:

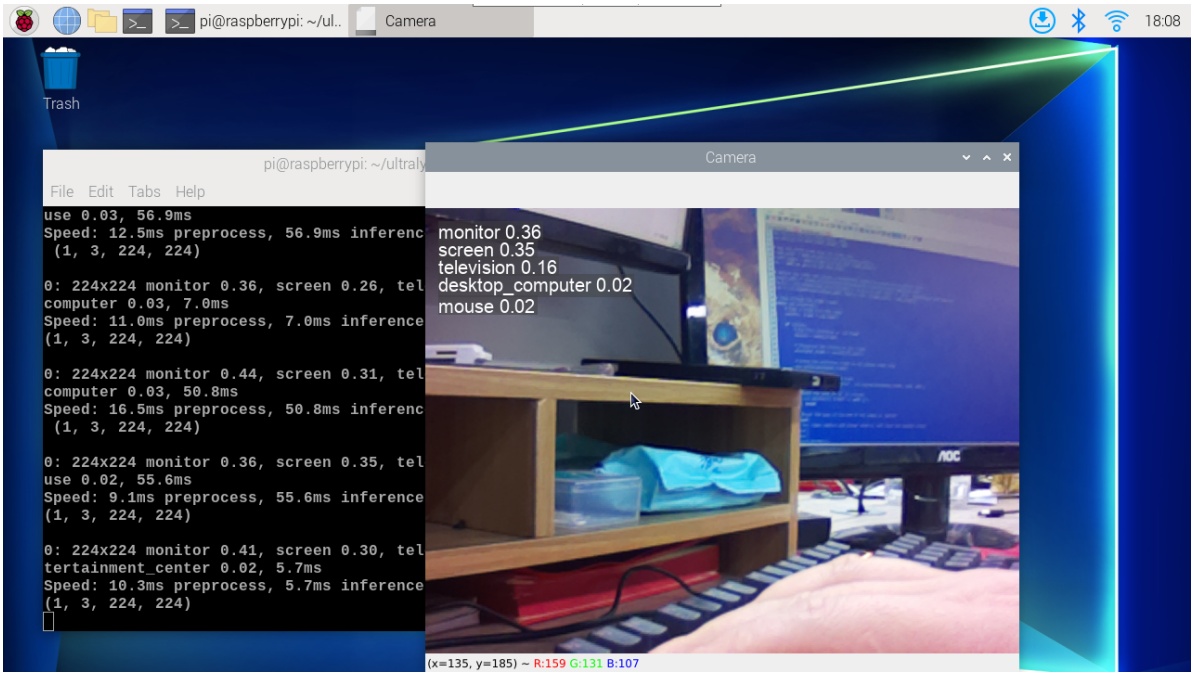
```
cd /home/pi/ultralytics26/yahboom_demo
```

Run the code:

```
python3 04.classification_camera_csi.py
```

### Effect Preview

yolo recognition output video location: /home/pi/ultralytics26/output/




Example code:

```
import cv2
from picamera2 import Picamera2

from ultralytics import YOLO

# Initialize the Picamera2
picam2 = Picamera2()
picam2.preview_configuration.main.size = (640, 480)
picam2.preview_configuration.main.format = "RGB888"
picam2.preview_configuration.align()
picam2.configure("preview")
picam2.start()

# Load the YOLO model
#model = YOLO("/home/pi/ultralytics26/yolo26n-cls.pt")
model = YOLO("/home/pi/ultralytics26/yolo26n-cls_ncnn_model")

# Set up video output
output_path = "/home/pi/ultralytics26/output/04.classification_camera_csi.mp4"
```

```
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter(output_path, fourcc, 30, (640, 480))

while True:
    # Capture frame-by-frame
    frame = picam2.capture_array()

    # Run YOLO11 inference on the frame
    results = model(frame)

    # Visualize the results on the frame
    annotated_frame = results[0].plot()

    # Write the frame to the video file
    out.write(annotated_frame)

    # Display the resulting frame
    cv2.imshow("Camera", annotated_frame)

    # Break the loop if 'q' is pressed
    if cv2.waitKey(1) == ord("q"):
        break

# Release resources and close windows
picam2.close()
out.release()
cv2.destroyAllWindows()
```

## References

[Image Classification - Ultralytics YOLO Documentation]